TAKE-HOME DELIVERY

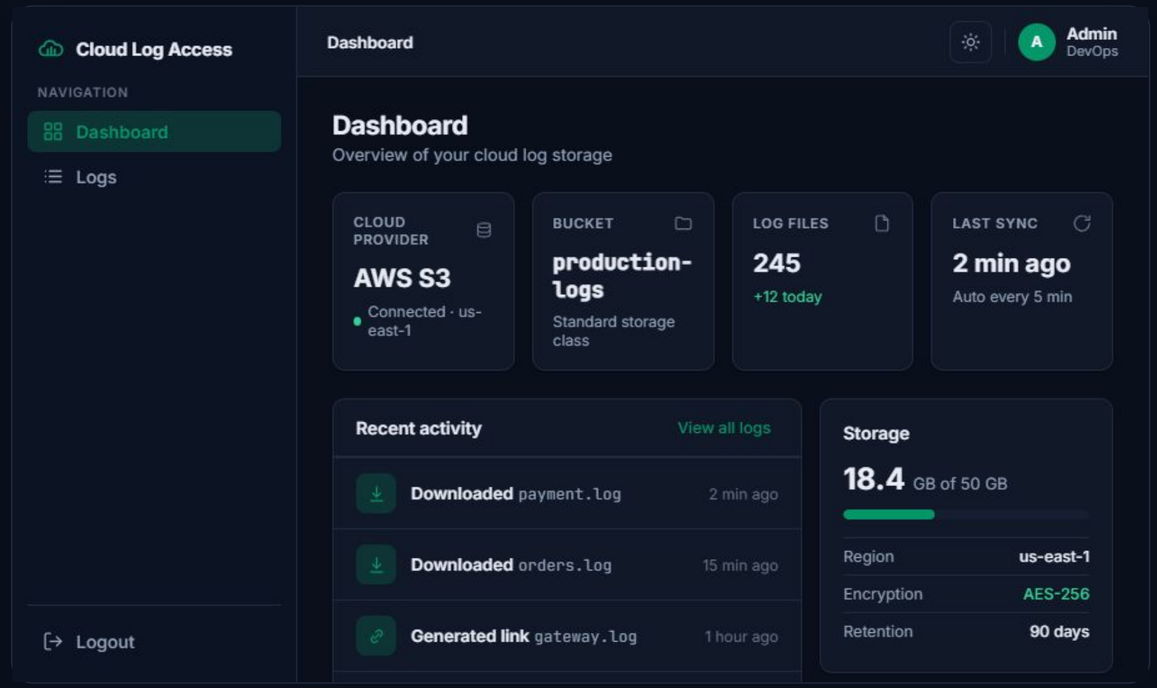
# Cloud Log Access Service

Secure, multi-cloud log access — a **Go BFF** + **React SPA** .

Role-controlled access to log files across AWS S3, GCP GCS, and Azure Blob.

**All required** **All 4 bonuses** **3 clouds**

Runs in one command — published Docker images, no toolchain.



LIVE WALKTHROUGH

# Live demo

1. **Admin login** → real S3 logs load in the dashboard
2. **Switch AWS → GCP → Azure** — same UI, three clouds
3. **Download** a file (streamed) and **generate a temporary link** — open it in a tab
4. **Log in as viewer** → AWS-only · GCP shows no-permission · temp-link button gone **(RBAC)**

DEMO CREDENTIALS

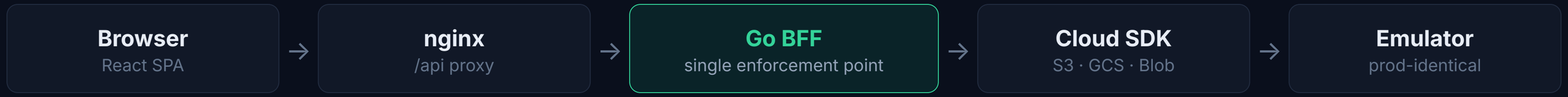
`admin@example.com` `admin123`

`viewer@example.com` `viewer123`

`localhost:8080`

ARCHITECTURE & BACKEND DESIGN

# One enforcement point, three thin adapters




## One `storage.Provider` interface

A Service facade owns allow-listing, key sanitization & TTL clamping. Adding a cloud = one ~100-line file.

```
type Provider interface  {
  List(ctx, bucket, prefix)   ([]Object,
error )
  Download(ctx, bucket, key)  (io.ReadCloser,
error )
  Presign(ctx, bucket, key, ttl) (
string, error )
}
```

## Dual-endpoint signing

Presigned URLs must open from the **browser** , but the BFF talks to internal hostnames. SigV4 / SAS sign the host — so I sign against `localhost` directly.

**Never rewrite a signed URL.**

SECURITY · AUTH & AUTHZ

# Defense in depth, every request

RequestID › Rate limit › **JWT auth** › **RBAC** › Sanitize › Handler

## JWT HS256 — algorithm-pinned

Blocks `alg:none` / confusion. `iss/aud/exp/nbf`, 1h TTL, jti denylist so logout truly revokes.

## bcrypt + constant-time auth

No user enumeration. RBAC admin/viewer → viewer scoped to AWS = `403 PROVIDER_NOT_ALLOWED`.

## Rate limiting & IDOR defense

Keyed on real client IP (`X-Real-IP`, not the proxy). Path-traversal sanitizer; presigned URLs never logged.

## Security-auditor pass

Ran an adversarial audit and implemented its MUST controls — OWASP-mapped, with table-driven tests.

FRONTEND · CONTAINERS · BONUSES

# Role-aware UI, zero-build deploy

## Frontend

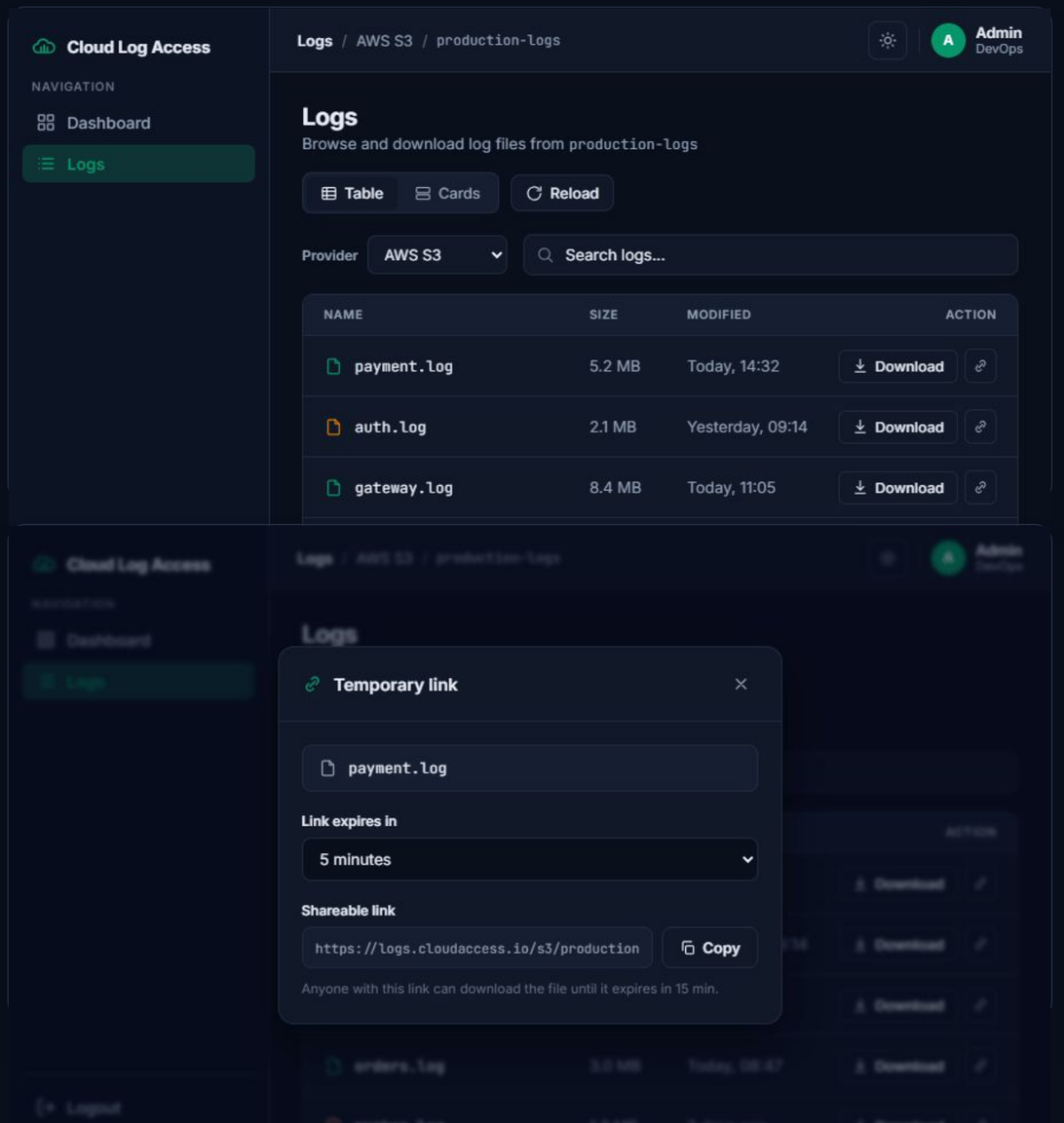
React 18 + TS + Tailwind · Zustand (persisted session) + TanStack Query · route guards · role-aware UI — hidden temp-link & no-permission state for viewers.

## Containers

Multi-stage distroless / non-root images. One command runs emulators + seeder + BFF + frontend. Published to Docker Hub → run with zero build.

## Bonuses

Terraform (S3 bucket + lifecycle + seed on LocalStack) · GitHub Actions (gofmt/vet/build, `go test -race` , LocalStack smoke, image builds) · docker-publish pipeline.



DECISIONS, TRADE-OFFS & CLOSE

# Deliberate choices, honest trade-offs

| DECISION | WHY |
| --- | --- |
| **Go** | Streaming `io.Copy`, first-party SDKs, static distroless binary |
| **JWT + RBAC** | Robust yet simple, stateless — right-sized vs. full OIDC |
| **Emulators** | Production-identical code — swap one endpoint for real cloud |
| **Self-review** | Adversarial pass caught & fixed real issues; clean history |

**PRODUCTION PATH**

GCS real V4 signing (SA key) · httpOnly cookies vs localStorage · persistent users + OIDC · Redis-backed denylist · cloud IAM least-privilege

`github.com/pedrohrbispo/cloud-logs-service`

Images on Docker Hub — happy to dive into any part of the code.